Straggler-Tolerant Federated Learning via Systematic Erasure Coding

Md Khairul Azman
Department of Computer Science
Texas State University
San Marcos, Texas

Mahfujur Rahman
Department of Computer Science
Texas State University
San Marcos, Texas

Tanzima Islam
Department of Computer Science
Texas State University
San Marcos, Texas

***Abstract*—Stragglers—slow or unresponsive clients—are a key bottleneck in Federated Learning (FL), causing synchronous rounds to stall. Inspired by the coding-theoretic framework of Tandon et al. [2], we adapt erasure coding to the federated setting, where parity clients hold backup data partitions for straggler-prone peers, ensuring aggregation always covers the full data distribution. We implement this alongside four baseline strategies—full synchronous, partial participation, deadline-based, and asynchronous—using the Flower framework [3] on a simulated 100-client non-IID FashionMNIST setup. Results show that erasure coding maintains accuracy competitive with the synchronous baseline while tolerating up to 10 stragglers per round, with straggler delay and centralized evaluation identified as the dominant sources of overhead.**

***Index Terms*—Federated Learning, Straggler Mitigation, Erasure Coding, Gradient Coding, Non-IID Data, Communication Efficiency, Flower Framework**

I. Introduction

Federated Learning (FL) has emerged as a prominent paradigm for training machine learning models across distributed devices without centralizing raw data [1], [12], [13]. Its applications span mobile keyboard prediction [27], healthcare [28], and IoT systems [29], where data privacy and communication constraints make centralized training infeasible.

Stragglers are a fundamental challenge in distributed and federated learning. In synchronous FL, the server must wait for all participating clients to return their model updates before aggregating and advancing to the next round. When even a small number of clients are slow—due to hardware heterogeneity, network congestion, or resource contention—the entire round is delayed. This problem is well documented in distributed computing [4]–[6] and becomes especially pronounced in FL, where clients are commodity devices with unpredictable availability [17].

The standard approach, Federated Averaging (FedAvg) [1], operates as follows. A server broadcasts a global model to N clients. Each client trains locally on its own data partition and returns a model update. The server aggregates all updates and advances to the next round. The bottleneck is that the round cannot complete until the slowest client responds, making wall-clock time proportional to the worst-case client delay [19].

Several practical workarounds exist. Partial participation selects only a fraction K of clients per round, naturally excluding slow nodes [1]. Deadline-based aggregation closes the round after a fixed timeout and uses only the updates that arrived in time [17]. Asynchronous-style aggregation keeps only the fastest K responses within a short time window [18]. While these approaches reduce wait time, they discard updates from slow clients, which can hurt convergence when data is non-IID [14], [30], since excluded clients may hold under-represented classes or features that never contribute to training.

Inspired by the gradient coding framework of Tandon et al. [2], we propose an erasure-coded FL strategy that avoids this tradeoff. We assign designated parity clients to hold backup data partitions for straggler-prone peers. If a slow client misses the round deadline, its parity client automatically carries a trained update over the missing partition. The server-side aggregation therefore always covers the full data distribution, with no accuracy loss relative to the synchronous baseline and no need for coordination between clients.

We implement this scheme alongside four baseline strategies using the Flower federated learning framework [3], simulating 100 clients on a non-IID FashionMNIST [11] setup. We compare all strategies on accuracy, training loss, communication cost, and per-phase compute overhead. Our results show that erasure coding tolerates up to 10 stragglers per round while preserving model quality, at the cost of a modest compute overhead on parity clients.

A. The Effects of Stragglers

In our simulated setup, we model straggler behavior by injecting random delays into a designated subset of clients. Clients 0 to 9 (out of 100) are marked as straggler-prone and receive delays sampled uniformly between 2 and 5 seconds per round, while the remaining clients complete within 0 to 0.6 seconds. This simulates the real-world observation that a small fraction of slow workers can delay an entire synchronous round by several times the typical response time [19], [31].

B. Related Work

a) Straggler mitigation in distributed learning.: Several approaches have been proposed to address the straggler problem in distributed learning. Asynchronous methods [4], [5], [18] avoid waiting for slow workers but sacrifice convergence guarantees and reproducibility. Chen et al. [7] use synchronous SGD with backup workers to maintain minibatch size, but

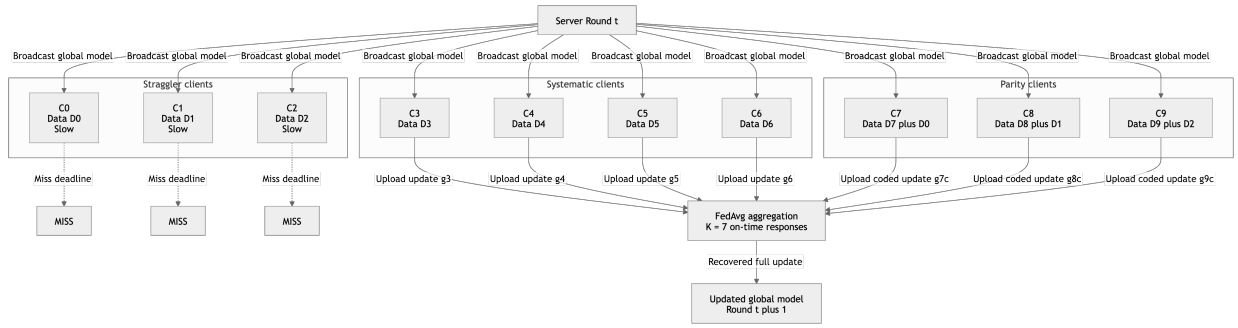


Fig. 1: Overview of the proposed erasure-coded federated learning framework. The server broadcasts the global model to all clients. Straggler clients fail to meet the deadline, while systematic and parity clients upload their updates on time. Using coded redundancy, the server aggregates a subset of responses to reconstruct the full global update without accuracy loss.

consistently slow clients may never contribute to training. Narayanamurthy et al. [8] approximate the loss on failed partitions using a linear expansion, which is only valid near the last known iterate. Reisizadeh et al. [19] propose a coded computation approach specifically for FL, showing provable tolerance to a fixed number of stragglers.

b) Coding-theoretic approaches.: Most relevant to our work, Lee et al. [9] and Dutta et al. [10] apply coding theory to distributed matrix multiplication and data shuffling, treating stragglers as erasures. Tandon et al. [2] extend this idea directly to gradient computation, showing that coding across gradient vectors can recover the full batch gradient with any fixed number of stragglers. Yu et al. [24] further generalize gradient coding using polynomial codes. Our work adapts this gradient coding principle to the federated learning setting using the Flower framework [3].

c) Non-IID data and heterogeneity.: Non-IID data distribution is a central challenge in FL [14], [32]. When client data distributions differ significantly, partial participation and straggler dropout can cause the global model to drift [16], [30]. SCAFFOLD [16] addresses client drift using control variates, while FedProx [15] adds a proximal term to local objectives to limit divergence from the global model.

d) Communication efficiency.: Communication cost is a well-recognized bottleneck in FL [20]. Prior work has proposed gradient compression [21], quantization [22], and sparsification [23] to reduce the volume of transmitted data. Our erasure-coded scheme incurs no additional communication overhead relative to the synchronous baseline, since parity clients transmit a single averaged update.

e) Privacy and security in FL.: Beyond efficiency, FL systems must contend with privacy risks. Differential privacy mechanisms [25], [26] can be combined with FL to provide formal guarantees, and our scheme is compatible with such approaches as a direction for future work.

II. Preliminaries

In federated learning, N clients collectively train a shared model without exchanging raw data [12]. Each client i holds a local data partition D_i, and the global objective is:

$$\beta^* = \arg\min_{\beta} \sum_{i=1}^{N} \frac{|D_i|}{|D|} \mathcal{L}(\beta; D_i) \tag{1}$$

where $\mathcal{L}(\cdot)$ is the local loss function (cross-entropy in our setting), and the weighting is proportional to partition size. The server updates the global model via FedAvg [1]:

$$\beta^{(t+1)} = \sum_{i=1}^{N} \frac{|D_i|}{|D|} \beta_i^{(t)} \tag{2}$$

where $\beta_i^{(t)}$ is the locally trained model returned by client i in round t. Under non-IID data, the weighted average may not converge to the true optimum when client updates are computed on heterogeneous distributions [14], [15]. The bottleneck arises when slow clients delay the aggregation [19].

A. Notation

Throughout this paper, N denotes the total number of clients, K the minimum number of responses required to proceed, and $S = N - K$ the straggler tolerance. Clients are denoted $C_0, C_1, \ldots, C_{N-1}$. The local model update from client i is denoted β_i. Systematic clients hold one primary partition each, while parity clients hold one primary and one backup partition. We use $N = 100$, $K = 90$, and $S = 10$ in all experiments.

B. The General Setup

We extend the standard FedAvg setup to tolerate S stragglers by introducing parity clients, following the spirit of coded distributed computation [2], [24]. Let the set of straggler-prone clients be

$$\mathcal{P} = \{0, 1, \ldots, 9\}$$

and the set of parity clients be

$$\mathcal{Q} = \{90, 91, \ldots, 99\},$$

with a fixed backup mapping $\phi : \mathcal{Q} \to \mathcal{P}$ defined as $\phi(90 + i) = i$.

Each parity client $j \in \mathcal{Q}$ trains on both its primary partition D_j and its assigned backup partition $D_{\phi(j)}$, returning a weighted average update:

$$\beta_j = \frac{|D_j|}{|D_j| + |D_{\phi(j)}|}\beta_j^{\text{primary}} + \frac{|D_{\phi(j)}|}{|D_j| + |D_{\phi(j)}|}\beta_j^{\text{backup}}. \quad (3)$$

If any client $i \in \mathcal{P}$ misses the round deadline, its corresponding parity client $j = 90 + i$ already carries a trained update over D_i. The server-side FedAvg aggregation therefore always covers all data partitions, with no changes required to the aggregation logic.

The rest of this paper is organized as follows. Section III describes the five strategies implemented and their configurations. Section IV presents experimental results comparing accuracy, communication cost, and compute overhead. Section V discusses limitations, and Section VI concludes.

III. Methodology

A. Federated Averaging with Straggler Tolerance

Standard FedAvg [1] aggregates updates from all N clients in each round. Let $\beta_i^{(t)}$ denote the local model returned by client i after training on partition D_i in round t. The global update is:

$$\beta^{(t+1)} = \sum_{i=0}^{N-1} \frac{|D_i|}{|D|}\beta_i^{(t)}. \quad (4)$$

If any client i fails to respond before the round closes, its partition D_i contributes nothing to $\beta^{(t+1)}$. Under non-IID data, this causes systematic bias [14], as the global model drifts away from the true data distribution. Strategies such as FedProx [15] and SCAFFOLD [16] address drift but do not directly solve the straggler problem.

B. Systematic Erasure Coding for FL

We partition the N clients into two disjoint sets, following the systematic erasure code construction of [2], [10]. Systematic clients

$$\mathcal{S} = \{0, 1, \ldots, N - S - 1\}$$

each hold one unique primary partition and behave identically to standard FedAvg clients. Parity clients

$$\mathcal{Q} = \{N - S, \ldots, N - 1\}$$

each hold one primary partition and one designated backup partition, with a fixed mapping:

$$\phi : \mathcal{Q} \to \mathcal{P}, \quad \phi(N - S + i) = i, \quad i = 0, 1, \ldots, S - 1, \quad (5)$$

where $\mathcal{P} = \{0, 1, \ldots, S - 1\}$ is the set of straggler-prone clients.

C. Parity Client Update

Each parity client $j \in \mathcal{Q}$ trains two local models from the same global initialization $\beta^{(t)}$: one on its primary partition D_j and one on its backup partition $D_{\phi(j)}$. It returns a weighted average:

$$\beta_j^{(t)} = \frac{|D_j|}{|D_j| + |D_{\phi(j)}|} \cdot \beta_j^{\text{primary}} + \frac{|D_{\phi(j)}|}{|D_j| + |D_{\phi(j)}|} \cdot \beta_j^{\text{backup}}. \quad (6)$$

This single transmission carries information from two data partitions while keeping the communication cost identical to that of a standard client, in contrast to naive data replication strategies that would double communication overhead [20].

D. Server-Side Aggregation

Let $\mathcal{R}^{(t)} \subseteq \{0, \ldots, N-1\}$ be the set of clients that respond within the round deadline. The server aggregates using FedAvg over responding clients:

$$\beta^{(t+1)} = \sum_{i \in \mathcal{R}^{(t)}} \frac{w_i}{\sum_{k \in \mathcal{R}^{(t)}} w_k} \cdot \beta_i^{(t)}, \quad (7)$$

where $w_i = |D_i|$ for systematic clients and $w_j = |D_j| + |D_{\phi(j)}|$ for parity clients. No matrix inversion or special decoding step is required at the server, unlike in some coded computation frameworks [9], [24].

E. Fault Tolerance Guarantee

If any subset $\mathcal{F} \subseteq \mathcal{P}$ of straggler-prone clients fails, where $|\mathcal{F}| \leq S$, then by construction each failed client $i \in \mathcal{F}$ has a corresponding parity client $j = N - S + i \in \mathcal{Q}$ whose response already encodes a trained update over D_i. The aggregation therefore covers all N data partitions regardless of which clients in $\mathcal{P}$ are slow [2].

Formally, the union of partitions represented in $\mathcal{R}^{(t)}$ satisfies:

$$\bigcup_{i \in \mathcal{R}^{(t)}} \text{partitions}(i) = \{D_0, D_1, \ldots, D_{N-1}\} \quad \text{whenever } |\mathcal{F}| \leq S. \quad (8)$$

In our 10-node experiments, we use $N = 10$, $K = 7$, and $S = 3$, with parity clients 7, 8, 9 backing up straggler-prone clients 0, 1, 2, respectively.

IV. Experiments

In this section, we present experimental results comparing our proposed erasure-coded FL strategy against four baseline approaches. We evaluate each strategy on final model accuracy, training loss, communication cost, and total round time under simulated straggler conditions.

A. Experimental Setup

All methods were implemented in Python using the Flower federated learning framework [3]. Experiments were run locally on a single machine using Flower's simulation engine, with 10 clients and 5 communication rounds. Each client holds a non-IID partition of the FashionMNIST dataset [11], created by sorting samples by class label, dividing them into

shards, and assigning 2 shards per client—a standard non-IID construction used widely in FL benchmarks [1], [14]. The model is a two-layer CNN trained with SGD (learning rate 0.01, momentum 0.9) for 1 local epoch per round.

Straggler behavior is simulated by injecting random delays: clients 0 to 2 receive delays sampled uniformly from $[2, 5]$ seconds, while the remaining clients complete within $[0, 0.6]$ seconds. We compare five strategies:

1) **Baseline**: waits for all clients [1];
2) **Partial** ($k = 5$): uses only 5 of 10 clients per round [1];
3) **Deadline (8s)**: closes the round after a fixed 8-second timeout [17];
4) **Async (1.5s)**: uses only the fastest responses within a 1.5-second window [18];
5) **Erasure Coded**: our proposed systematic $(10, 7)$ scheme tolerating up to 3 stragglers with no data loss [2].

B. Results

We evaluate each strategy across five components: model accuracy and loss, straggler delay, communication cost, client participation, and per-phase compute overhead.

1) Model Accuracy and Training Loss: Centralized accuracy measures how well the global model performs on the held-out FashionMNIST test set after each round of aggregation. Training loss reflects the average cross-entropy loss computed locally across all participating clients before sending updates. Together, these metrics indicate whether the aggregated model is converging and generalizing correctly [13].

The baseline reaches the highest final accuracy of 0.4975 with a loss of 1.2925 after 5 rounds, as it always aggregates all 10 client updates. Erasure coding achieves 0.4766, within 2.1 percentage points of the baseline, since parity clients ensure no data partition is ever excluded. Deadline (8s) reaches 0.4429 but wastes round 1 entirely due to all straggler-prone clients missing the cutoff. Partial participation reaches only 0.4188 because the same 5 clients are excluded every round, creating a persistent bias under non-IID data [14]. Async (1.5s) fails completely, remaining at its initial accuracy of 0.1103 across all rounds, as the 1.5-second window is too short to collect any updates from straggler-prone clients—consistent with known convergence instability of aggressive asynchronous schemes [18].

2) Straggler Delay: Average straggler delay is the mean simulated wait time (in seconds) introduced by slow clients per round. Clients 0 to 2 are assigned delays drawn from $\text{Uniform}(2, 5)$ seconds, while the remaining clients complete within $\text{Uniform}(0, 0.6)$ seconds. This metric reflects how much each strategy is affected by slow clients before the server can proceed to aggregation [19].

The baseline incurs average delays of 0.72 to 1.61 seconds across rounds, as it always waits for the slowest client. Erasure coding shows a low delay of 0.49 seconds in round 2 (when only 5 clients respond) and stabilizes around 1.5 seconds in later rounds as all clients begin responding in time. Partial participation shows variable delays (0.43 to 1.95 seconds) depending on which 5 clients are selected each round.

3) Communication Cost: Communication cost is measured as the total bytes transmitted by all clients to the server per round, reported in megabytes. Each model parameter is a 32-bit float (4 bytes), so a full set of 10 client updates costs 4.23 MB per round. Strategies that aggregate fewer clients reduce this proportionally. This is consistent with the communication-efficiency framing of FL [20], [21].

The baseline, deadline, and erasure-coded strategies all transmit 4.23 MB per completed round since they aggregate all 10 clients. Partial participation transmits 2.12 MB per round as only 5 clients respond. Async transmits 0 MB across all rounds since no client meets the 1.5-second window. Erasure-coded parity clients send a single model update despite encoding two partitions, keeping per-round communication identical to the baseline—a key advantage over naive replication [22].

4) Client Participation and Failures: This component tracks how many clients successfully contribute updates each round and how many fail or are dropped. It directly reflects the straggler tolerance of each strategy. In large-scale FL deployments, reliable participation across heterogeneous devices is a persistent challenge [13], [17].

The baseline records zero failures across all rounds but only by waiting indefinitely. Deadline and erasure-coded strategies both record 10 failures in round 1, meaning all straggler-prone clients missed the cutoff in that round. From round 2 onward, erasure coding recovers full participation (10 clients) with zero failures. Async records 10 failures in every round, never achieving any successful aggregation. Partial participation records zero failures by design, since it only expects 5 clients to respond.

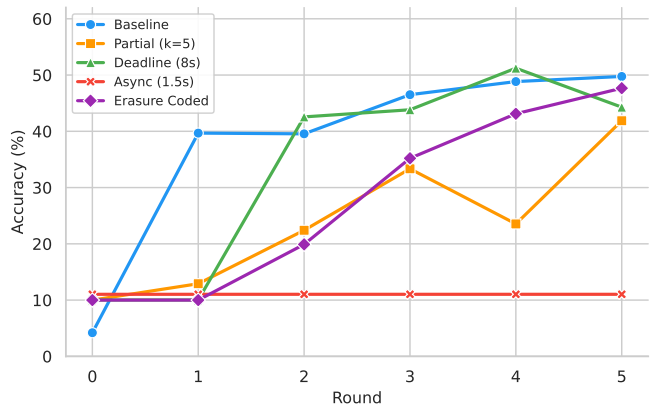
5) Per-Phase Compute Overhead: Each training round is decomposed into eight measurable phases. The following analysis is based on the erasure-coded strategy, as it is our primary contribution. Figure 4 shows the stacked breakdown per round, and Table I summarizes the average time and proportion of each phase across all rounds.

a) Data Load I/O.: This phase measures the time to fetch each mini-batch from disk into memory. In our experiments, this cost is effectively 0.000s (0.0% of total), since the FashionMNIST dataset fits entirely in RAM after the first load and subsequent accesses are served from cache.

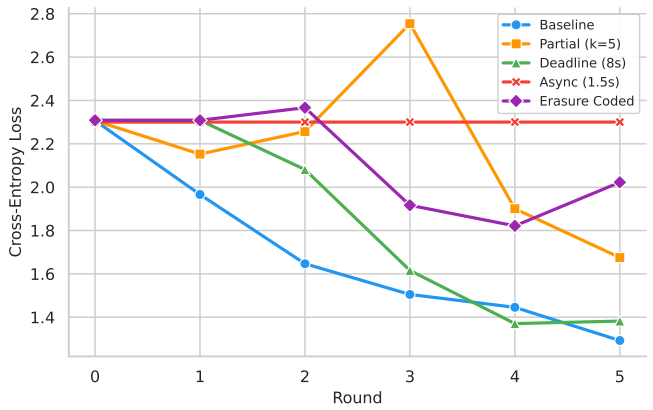
b) Forward Pass.: This is the time required to compute predictions by passing a batch through the CNN. It averages 0.040s per round (3.3% of total), reflecting the relatively small size of our two-layer CNN on CPU.

c) Backward Pass.: This phase measures the time to compute gradients via backpropagation. It averages 0.068s per round (5.7% of total), slightly higher than the forward pass due to the additional computation required to propagate gradients through each layer.

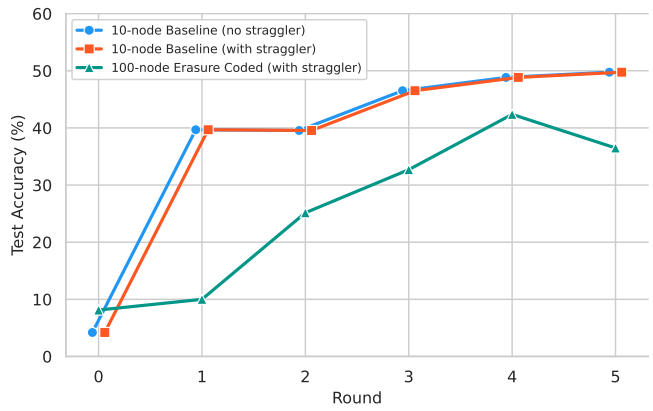
d) Serialization.: Serialization corresponds to converting model weight tensors from PyTorch format into NumPy arrays for transmission. It averages 0.001s (0.1% of total), confirming that this step introduces negligible overhead, consistent with findings in [3].



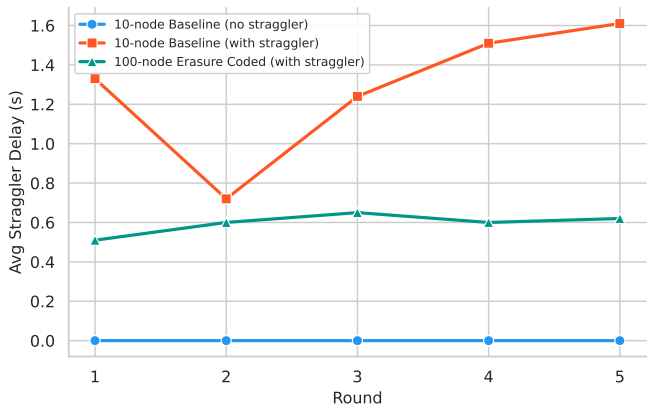
(a) Test accuracy per round



(b) Test loss per round



(c) Accuracy: 10-node vs. 100-node



(d) Straggler delay: 10-node vs. 100-node

Fig. 2: Convergence results across strategies and scales.

e) Compression.: This phase covers optional int8 quantization of parameters before transmission [22]. It averages 0.000s (0.0% of total), as compression was disabled in these runs.

f) Straggler Delay.: Straggler delay is the dominant bottleneck, averaging 0.597s per round and accounting for 50.3% of total round time. This corresponds to the simulated delay injected into straggler-prone clients (0 to 2), ranging from 0.51s in round 1 to 0.65s in round 3. This confirms that slow clients are the primary source of latency in synchronous FL [19], [31].

g) Aggregation.: Aggregation refers to the server-side FedAvg computation over received updates. It averages 0.021s (1.8% of total), which is negligible even for 10 clients, suggesting that aggregation will remain a minor cost as the number of clients scales [1].

h) Evaluation.: Evaluation is the centralized test inference run by the server at the end of each round. It averages 0.460s per round (38.8% of total), making it the second largest cost. This is primarily due to running full test-set evaluation every round, which would typically be performed less frequently in a production setting [17].

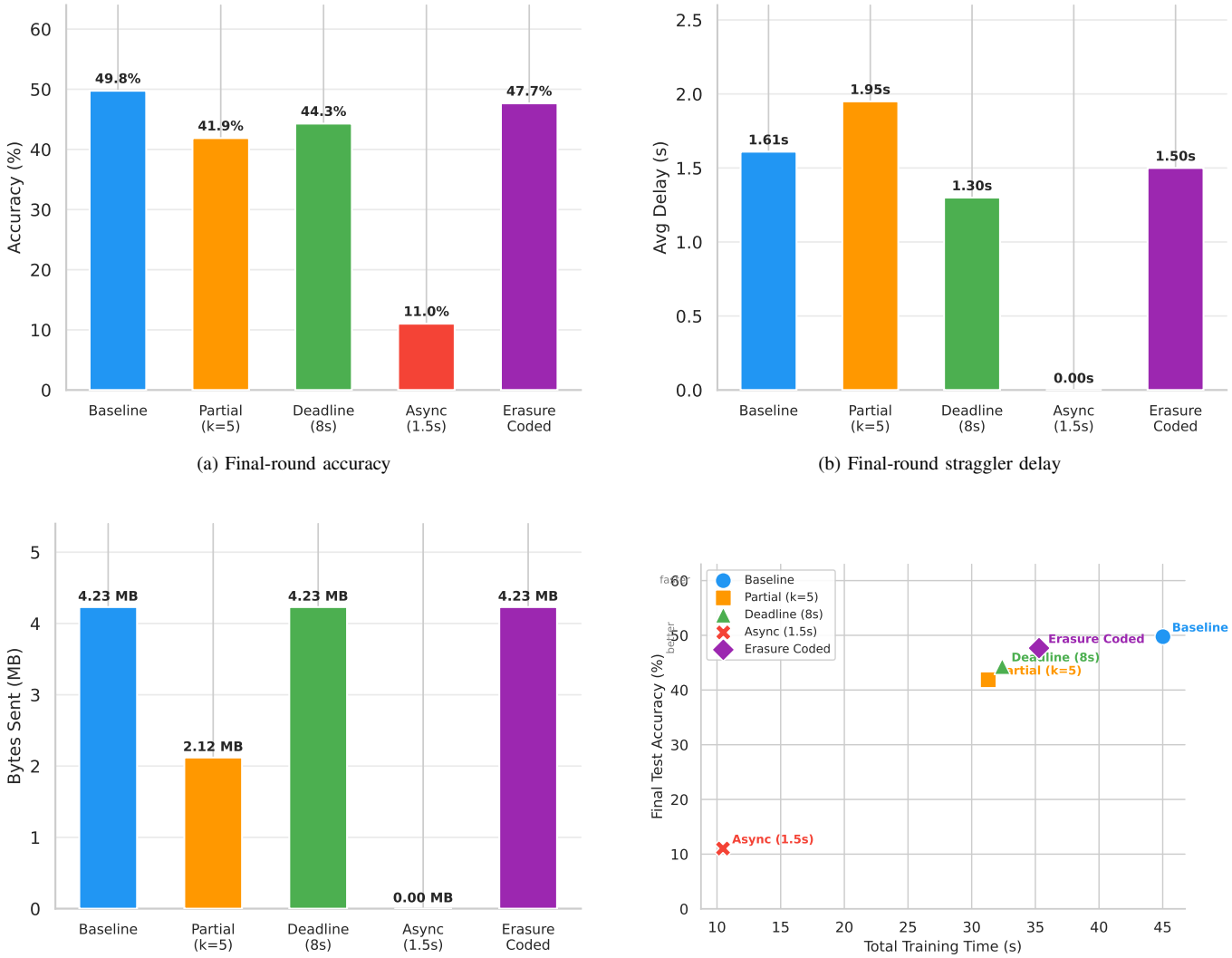
TABLE I: Average per-phase overhead for the erasure-coded strategy (10-node setting, 5 rounds).

Phase	Avg. Time (s)	% of Total
Data Load I/O	0.000	0.0%
Forward Pass	0.040	3.3%
Backward Pass	0.068	5.7%
Serialization	0.001	0.1%
Compression	0.000	0.0%
Straggler Delay	0.597	50.3%
Aggregation	0.021	1.8%
Evaluation	0.460	38.8%

V. Limitations

While our results demonstrate the viability of erasure-coded straggler mitigation, several limitations remain. First, all experiments were conducted on a single local machine using Flower's simulation engine [3]. Client delays were injected artificially using `time.sleep()`, rather than arising from real network latency or hardware variability. As a result, straggler behavior is controlled and predictable, unlike in real distributed environments [17].

Second, the 100-node erasure-coded run achieved a final



(a) Final-round accuracy

(b) Final-round straggler delay

(c) Communication cost

(d) Accuracy vs. training time trade-off

Fig. 3: Final-round comparison of federated learning strategies across accuracy, straggler delay, communication cost, and the accuracy–training time trade-off.

accuracy of 36.5%, compared to 49.8% for the 10-node baseline. This gap is partly explained by increased non-IID heterogeneity across 100 clients [30] and the higher communication cost of 42.35 MB per round. However, a full convergence analysis at this scale was not completed within the scope of this project.

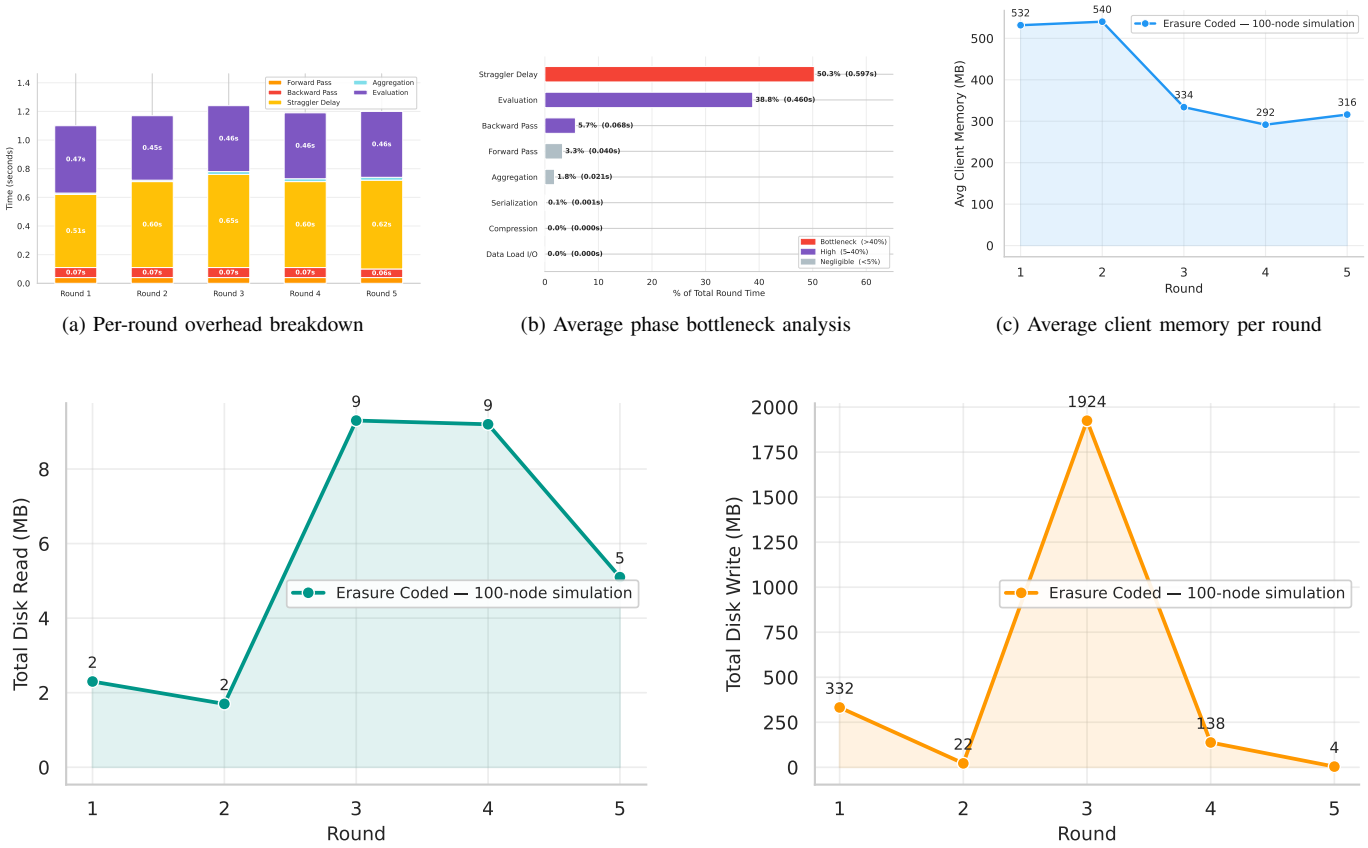
Third, the CPU utilization measured at 0.0% across all rounds reflects a limitation of the simulation environment. Flower executes clients sequentially within a single process, so per-client CPU measurements do not capture true parallel load as they would on real distributed hardware [13].

Finally, the erasure coding scheme protects only a fixed set of straggler-prone clients (clients 0 to 9 out of 100). In practice, any client may become a straggler [19], and assuming a known straggler set in advance is a strong constraint that limits real-world applicability.

VI. Conclusion

We proposed and implemented an erasure-coded straggler mitigation strategy for federated learning, inspired by gradient coding methods [2]. Parity clients hold backup data partitions for straggler-prone peers, ensuring that all partitions contribute to every aggregation round without requiring coordination between clients or modifications to the server-side FedAvg logic [1].

We evaluated the scheme at two scales. In the 10-node setup, erasure coding achieved 0.4766 accuracy, within 2.1 percentage points of the synchronous baseline (0.4975), while tolerating up to 3 stragglers per round with no data loss. This approach outperforms partial participation, deadline-based, and asynchronous alternatives in terms of convergence stability [14], [18]. Scaling to 100 nodes, erasure coding



(a) Per-round overhead breakdown

(b) Average phase bottleneck analysis

(c) Average client memory per round

(d) Total disk read per round

(e) Total disk write per round

Fig. 4: System-level overhead analysis for the 100-node erasure-coded setting, including per-round compute phases, average phase bottlenecks, client memory usage, and disk I/O.

reached 36.5% accuracy in 5 rounds, with an average straggler delay of 0.60s per round and 42.35 MB communication per round, tolerating up to 10 straggler clients simultaneously.

Per-phase overhead analysis confirms that straggler delay (50.3%) and centralized evaluation (38.8%) dominate total round time, while compute phases such as forward pass (3.3%), backward pass (5.7%), and serialization (0.1%) remain negligible.

VII. Future Work

Several directions remain open. Deploying the system on real distributed hardware, such as a university HPC cluster or cloud infrastructure, would replace simulated delays with real network and compute variability, providing a more realistic evaluation [17].

Closing the accuracy gap between the 100-node erasure-coded setup (36.5%) and the 10-node baseline (49.8%) is an important next step. Techniques such as FedProx [15] and SCAFFOLD [16] could help stabilize convergence under increased non-IID heterogeneity at scale.

Extending the erasure coding scheme to protect an arbitrary and dynamically changing subset of stragglers, rather than a fixed set known in advance, would significantly improve its applicability in real federated deployments [19].

Reducing evaluation overhead, which accounts for 38.8% of total round time, by running centralized evaluation less frequently or replacing it with federated evaluation [3] would improve overall system throughput.

Finally, incorporating differential privacy [25], [26] alongside erasure coding would allow the system to address both straggler tolerance and client data privacy simultaneously, which is a key requirement in real-world federated learning applications [12].

References

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist. (AISTATS)*, 2017, pp. 1273–1282.

[2] R. Tandon, Q. Lei, A. G. Dimakis, and N. Karampatziakis, "Gradient coding: Avoiding stragglers in distributed learning," in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 3368–3376.

[3] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, H. Li Kwing, T. Parcollet, P. P. B. de Gusmão, and N. D. Lane, "Flower: A friendly federated learning research framework," *arXiv preprint arXiv:2007.14390*, 2022.

[4] M. Li, D. G. Andersen, J. W. Park, A. J. Smola, A. Ahmed, V. Josifovski, J. Long, E. J. Shekita, and B.-Y. Su, "Scaling distributed machine learning with the parameter server," in *Proc. 11th USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*, 2014, pp. 583–598.

[5] Q. Ho, J. Cipar, H. Cui, S. Lee, J. K. Kim, P. B. Gibbons, G. A. Gibson, G. Ganger, and E. P. Xing, "More effective distributed ML via a stale synchronous parallel parameter server," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 26, 2013.

[6] J. Dean, G. Corrado, R. Monga, K. Chen, M. Devin, M. Mao, M. Ranzato, A. Senior, P. Tucker, K. Yang, Q. V. Le, and A. Ng, "Large scale distributed deep networks," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 25, 2012, pp. 1223–1231.

[7] J. Chen, R. Monga, S. Bengio, and R. Jozefowicz, "Revisiting distributed synchronous SGD," in *Proc. Int. Conf. Learn. Represent. (ICLR) Workshop*, 2016.

[8] S. Narayanamurthy and B. Taskar, "A nearly-linear time framework for graph-structured sparsity," in *Proc. 30th Int. Conf. Mach. Learn. (ICML)*, 2013, pp. 928–936.

[9] K. Lee, M. Lam, R. Pedarsani, D. Papailiopoulos, and K. Ramchandran, "Speeding up distributed machine learning using codes," *IEEE Trans. Inf. Theory*, vol. 64, no. 3, pp. 1514–1529, 2018.

[10] S. Dutta, V. Cadambe, and P. Grover, "Short-dot: Computing large linear transforms distributedly using coded short dot products," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 29, 2016.

[11] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.

[12] P. Kairouz et al., "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, no. 1–2, pp. 1–210, 2021.

[13] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Process. Mag.*, vol. 37, no. 3, pp. 50–60, 2020.

[14] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-IID data," *arXiv preprint arXiv:1806.00582*, 2018.

[15] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst. (MLSys)*, 2020, pp. 429–450.

[16] S. P. Karimireddy, S. Kale, M. Mohri, S. J. Reddi, S. U. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. 37th Int. Conf. Mach. Learn. (ICML)*, 2020, pp. 5132–5143.

[17] K. Bonawitz, H. Eichner, W. Grieskamp, D. Huba, A. Ingerman, V. Ivanov, C. Kiddon, J. Konečný, S. Mazzocchi, B. McMahan, T. V. Overveldt, D. Petrou, D. Ramage, and J. Roselander, "Towards federated learning at scale: A system design," in *Proc. Mach. Learn. Syst. (MLSys)*, 2019.

[18] C. Xie, S. Koyejo, and I. Gupta, "Asynchronous federated optimization," *arXiv preprint arXiv:1903.03934*, 2019.

[19] A. Reisizadeh, S. Prakash, R. Pedarsani, and A. S. Avestimehr, "Coded federated learning," in *Proc. IEEE Int. Workshop Signal Process. Adv. Wireless Commun. (SPAWC)*, 2020.

[20] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," *arXiv preprint arXiv:1610.05492*, 2016.

[21] Y. Lin, S. Han, H. Mao, Y. Wang, and B. Dally, "Deep gradient compression: Reducing the communication bandwidth for distributed training," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018.

[22] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "QSGD: Communication-efficient SGD via gradient quantization and encoding," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017.

[23] A. F. Aji and K. Heafield, "Sparse communication for distributed gradient descent," in *Proc. Conf. Empirical Methods Nat. Lang. Process. (EMNLP)*, 2017, pp. 440–445.

[24] Q. Yu, M. A. Maddah-Ali, and A. S. Avestimehr, "Polynomial codes: An optimal design for high-dimensional coded matrix multiplication," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017.

[25] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Found. Trends Theor. Comput. Sci.*, vol. 9, no. 3–4, pp. 211–407, 2014.

[26] R. C. Geyer, T. Klein, and M. Nabi, "Differentially private federated learning: A client level perspective," *arXiv preprint arXiv:1712.07557*, 2017.

[27] A. Hard, K. Rao, R. Mathews, S. Ramaswamy, F. Beaufays, S. Augenstein, H. Eichner, C. Kiddon, and D. Ramage, "Federated learning for mobile keyboard prediction," *arXiv preprint arXiv:1811.03604*, 2018.

[28] N. Rieke et al., "The future of digital health with federated learning," *NPJ Digit. Med.*, vol. 3, no. 1, p. 119, 2020.

[29] D. C. Nguyen, M. Ding, P. N. Pathirana, A. Seneviratne, J. Li, and H. V. Poor, "Federated learning for Internet of Things: A comprehensive survey," *IEEE Commun. Surv. Tutor.*, vol. 23, no. 3, pp. 1622–1658, 2021.

[30] K. Hsieh, A. Phanishayee, O. Mutlu, and P. Gibbons, "The non-IID data quagmire of decentralized machine learning," in *Proc. 37th Int. Conf. Mach. Learn. (ICML)*, 2020, pp. 4387–4398.

[31] S. Park, B. Han, and J. Moon, "SageFlow: Robust federated learning against both stragglers and adversaries," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 34, 2021.

[32] Q. Li, B. He, and D. Song, "Model-contrastive federated learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 10713–10722.